

Інтерактивна навчальна онлайн-платформа з вбудованим AI-асистентом на базі великих мовних моделей

Виконав:
студент групи КІСП-21
Гоцій М.М.
Керівник:
доцент кафедри ЕОМ
Бачинський Р.В.

Основна мета

Розробка інтерактивної онлайн-платформи для навчання з вбудованим AI-асистентом, яка поєднує роботу з навчальними матеріалами, автоматичну генерацію тестів та можливість виконання і перевірки програмного коду безпосередньо в браузері.

Параметри системи

Час відгуку векторного пошуку — до 10 мс

(досягається завдяки IVFFlat індексу в rgvector з 100 кластерами; прискорення у 6 разів порівняно з повним перебором).

Точність семантичного пошуку — понад 85% (precision@5)

(забезпечується OpenAI embeddings моделлю text-embedding-3-small; попередня обробка тексту з видаленням HTML-тегів)

Захист від зловживань: 99%+ ефективність блокування

(реалізовано Token Bucket алгоритм через Bucket4j + Caffeine Cache; ліміти: 5 спроб входу/хв, 10 реєстрацій/год, 3 скидання пароля/год).

Латентність запитів — 90-110мс

(забезпечується Spring Boot з оптимізованими JOIN FETCH запитами; індексацією бази даних; застосовано кешування через MapStruct DTO).

Актуальність теми

Зростання популярності онлайн-навчання

Недостатня інтерактивність існуючих платформ

Відсутність персоналізованої допомоги

Дослідження проблемної області виявило кілька ключових аспектів

Фрагментованість навчального процесу:

Сучасні онлайн-курси, тести та практичні завдання часто розміщені в різних системах. Користувачу доводиться постійно перемикатися між джерелами, що знижує концентрацію та ефективність навчання.

Обмежена інтерактивність навчального контенту:

Більшість існуючих платформ використовують статичні матеріали (текст, відео), які не адаптуються до контексту запиту користувача. Відсутність можливості миттєво отримати пояснення або підказку ускладнює самостійне навчання.

Обмеженість традиційних тестів:

Статичні набори тестових питань швидко втрачають ефективність через запам'ятовування відповідей. Це унеможлиблює об'єктивну перевірку знань і адаптацію складності. Використання AI дозволяє динамічно генерувати тести з різною кількістю питань і рівнем складності на задану тему.

Розглянуті аналоги

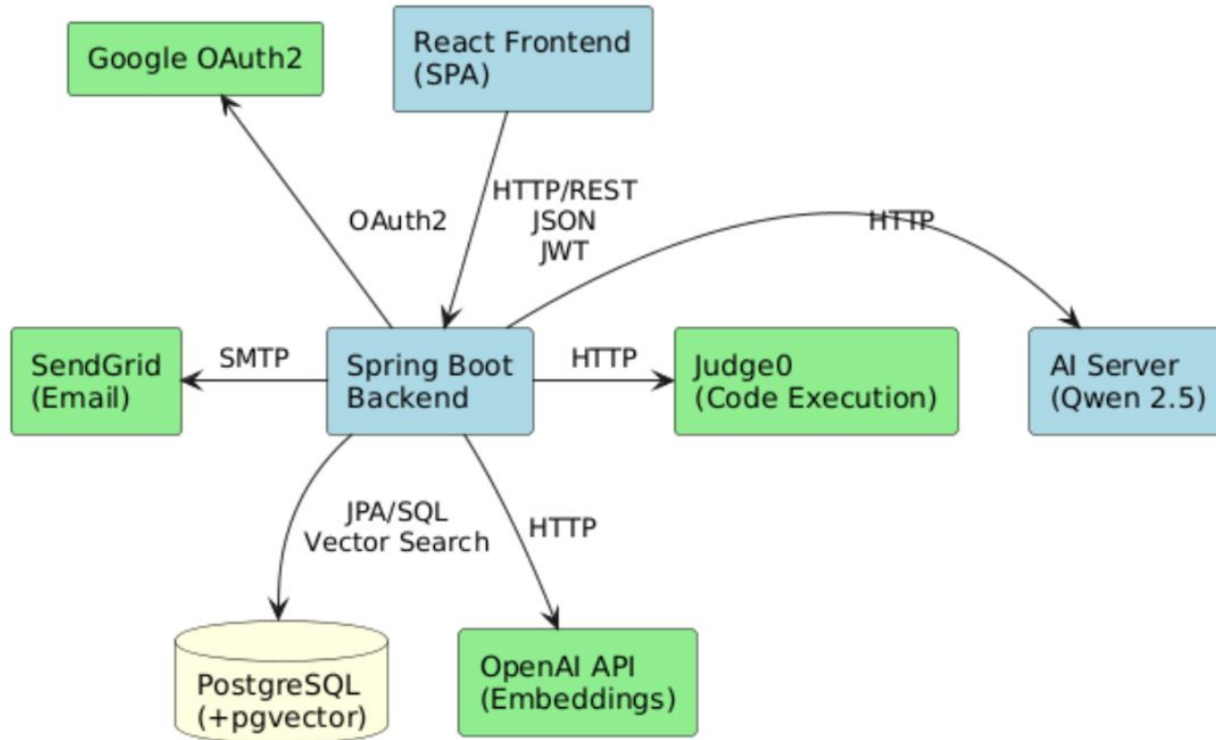
coursera



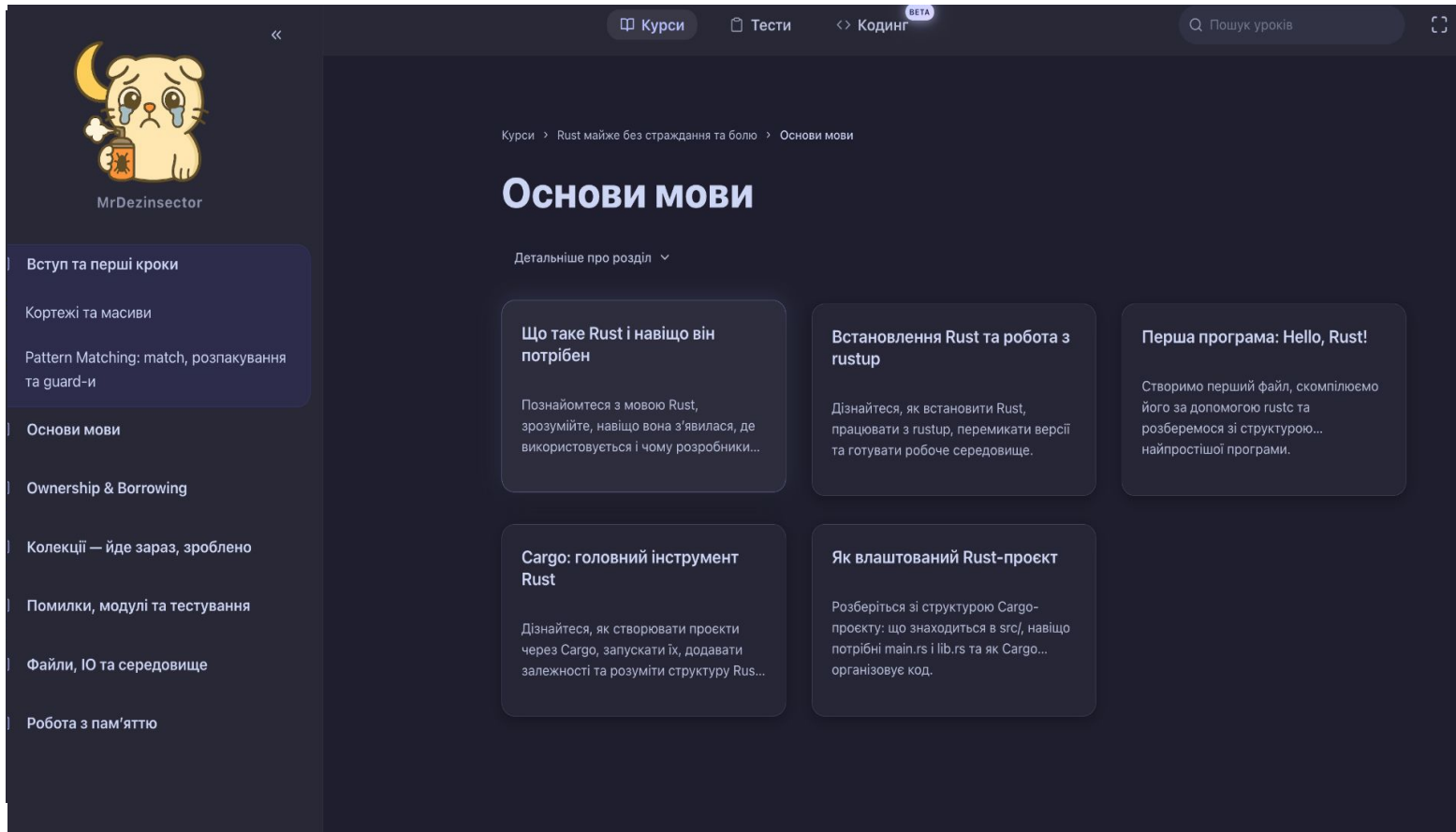
LeetCode

codecademy

Загальна архітектура системи



UI користувача-студента



UI Інтерактивні задачі та тести

Trinity

Курси Тести <> Кодинг

Пошук уроків

CA

ВВІД

а = -10, b = 20

ВІВІД

10

ПОЯСНЕННЯ

Від'ємне та додатне число: $-10 + 20 = 10$

Приклад 3:

ВВІД

а = 0, b = 0

ВІВІД

0

ПОЯСНЕННЯ

Сума двох нулів дорівнює нулю

Обмеження:

-1000 <= a, b <= 1000

Числа можуть бути від'ємними

Результат має бути цілим числом

Підказки ^

Це дуже просте завдання - просто використайте оператор +

Не забудьте повернути результат за допомогою return

main.rs Rust

▶ ↺ ↻ ↶ ↷

```
1 fn sum_two_numbers(a: i32, b: i32) -> i32 {
2     return a + b + 2;
3 }
```

> Вивід

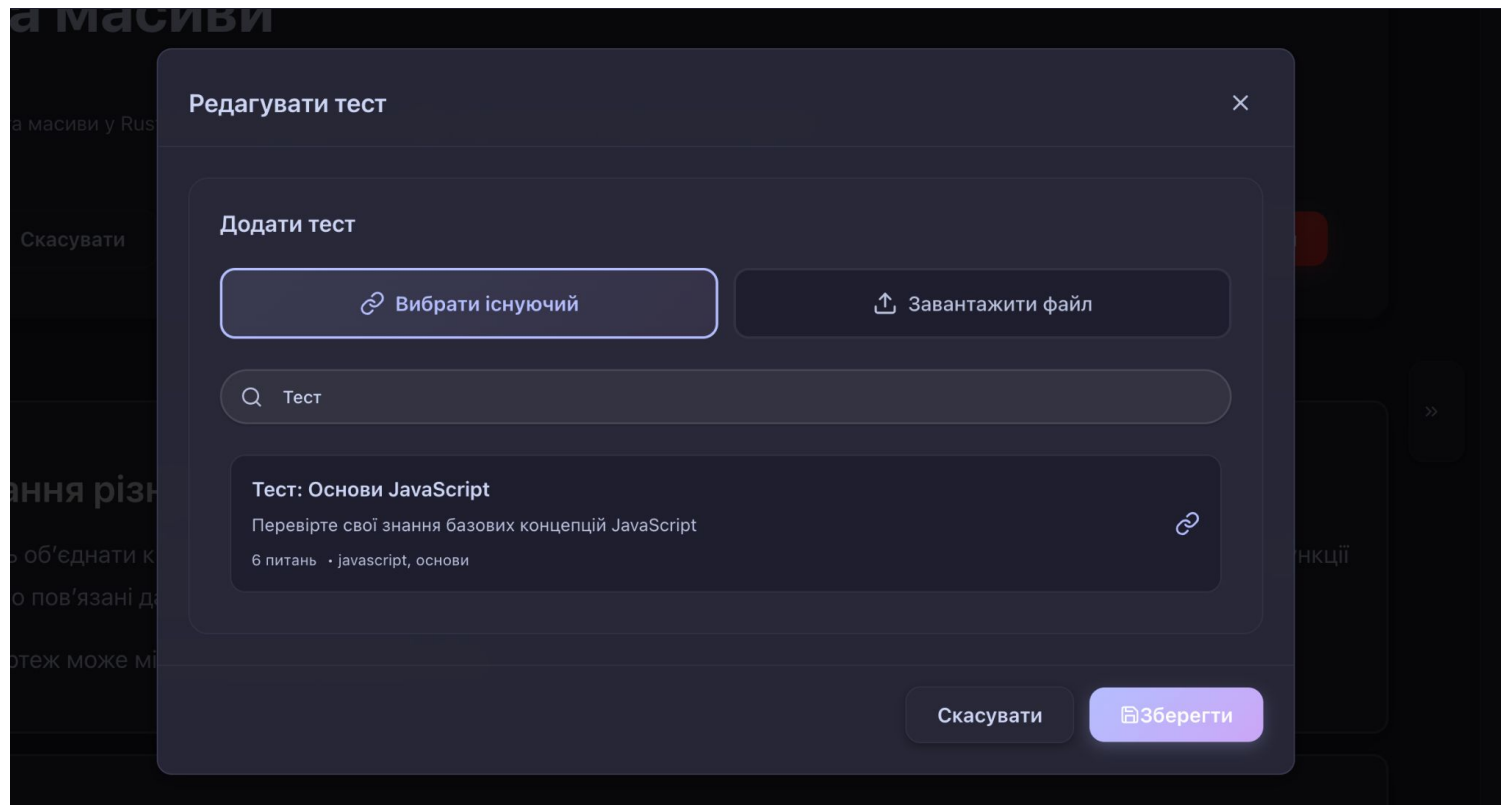
Історія спроб

Тест 1 0 ms

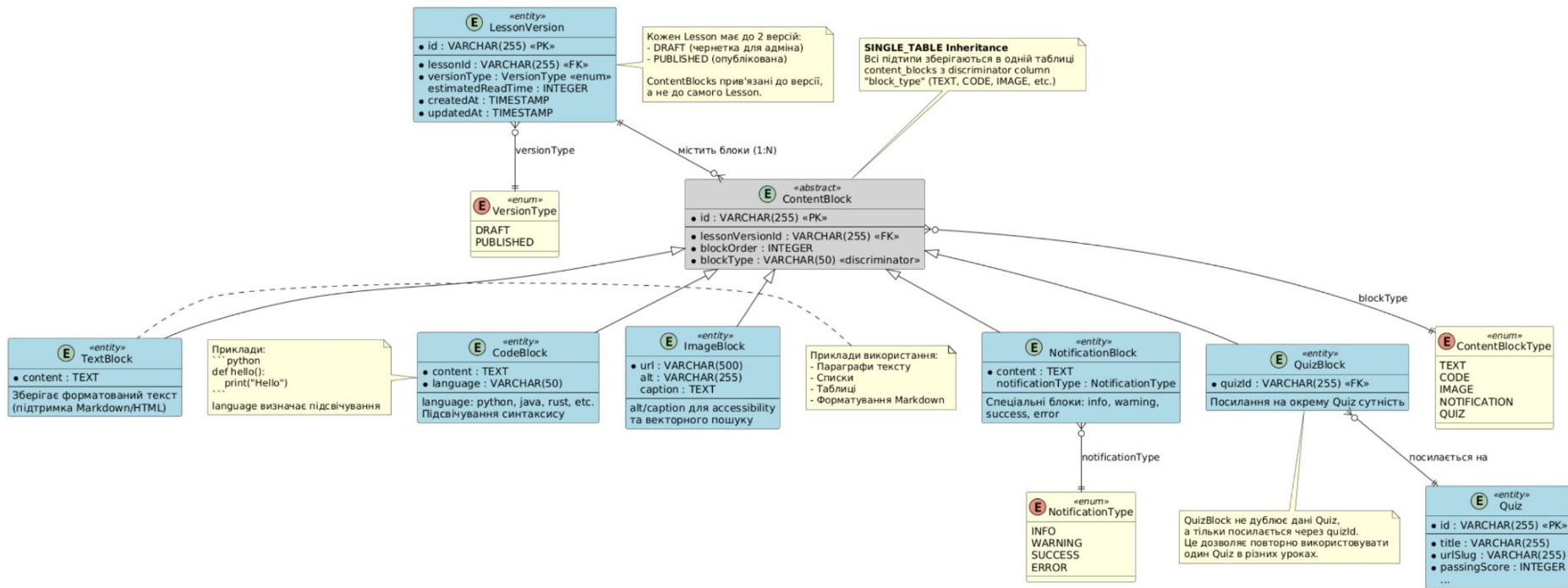
ВХІДНІ ДАНІ:
5, 3

ОТРИМАНО:
10

UI Адмін панель



ER-діаграма модульної системи контенту



AI-асистент: Qwen 2.5 VL-7B-Instruct

Ключові переваги:

Open-source — безкоштовна модель від Alibaba Cloud, повний контроль над даними

Мультимовність — підтримка 29+ мов, включно з українською та англійською

Оптимізація для коду — навчена на великому корпусі програмного коду (Python, Java, JavaScript, Rust)

Instruction-following — точне виконання структурованих завдань (пояснення, модифікація, генерація)

Великий контекст — 32,768 токенів (можна обробляти цілі файли коду)

Ефективність — 7B параметрів = баланс між якістю та швидкістю inference

AI-асистент



Trinity

до розділів

Ownership & Borrowing

Ownership у Rust: чому значення «кають», а не копіюються

Чому іноді потрібна move

Відключити від квартири

Чому це виглядає простіше, ніж

Чому не копіює String?

Чому значення копіюються, а не копіюються

Функції: значення «кає» в аргументи

Хто тримає коробку

Повернення значень

Милка: подвійний move

Ownership — серце Ownership

RUST

AI: Модифікувати приклад

```
1 fn make() -> String {
2     String::from("Rust")
3 }
4
5 fn main() {
6     let name = make();
7     println!("{}", name); // ✓ name тепер власник значення
8 }
```

Модифіковано код згідно з вашим запитом: with-error-handling

RUST

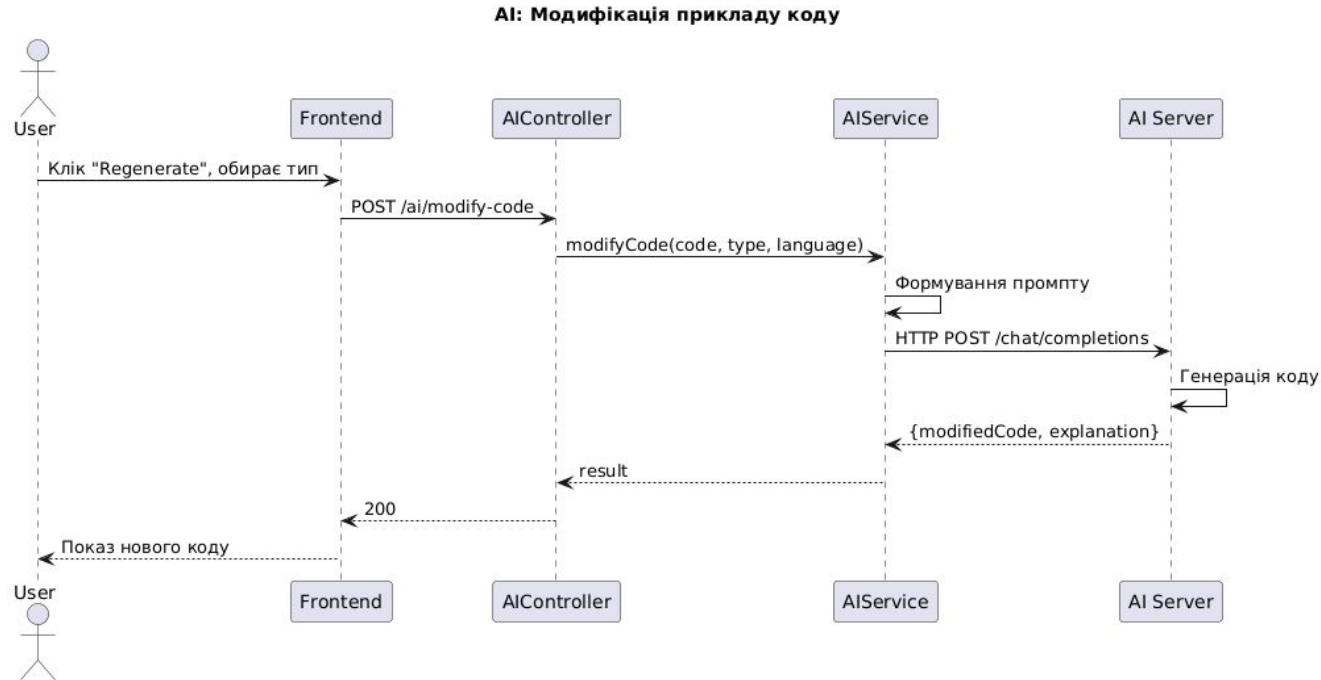
```
1 use std::error::Error;
2
3 fn create_strings() -> Result<(), Box<dyn Error>> {
4     let empty = String::new();
5     let name = String::from("Oleg");
6     let city = "Kyiv".to_string();
7
8     // Перевірка валідності
9     if name.is_empty() {
10         return Err("Name cannot be empty".into());
11     }
12
13     Ok(())
14 }
```

✓ Додано обробку помилок через Result<T, E>

✓ Обгорнуто код у функцію з перевітками

✓ Додано валідацію даних

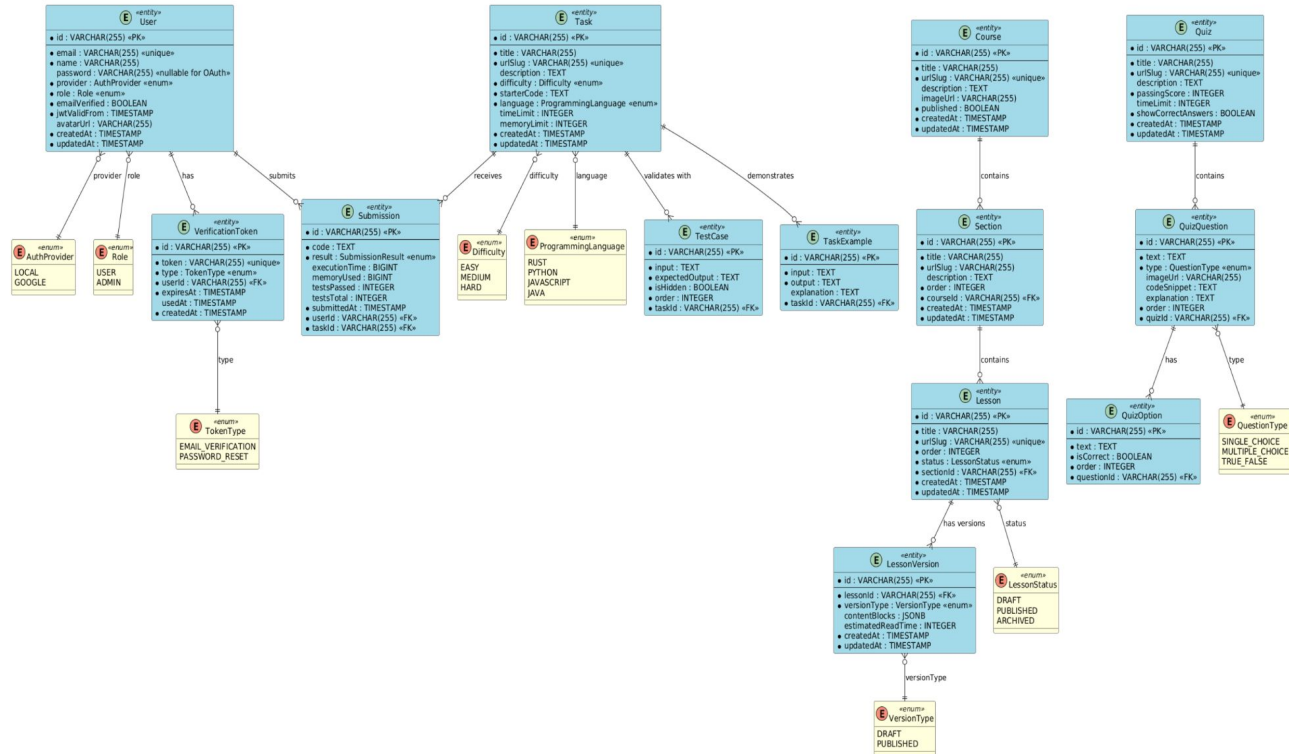
Діаграма послідовності модифікації коду з AI-асистентом



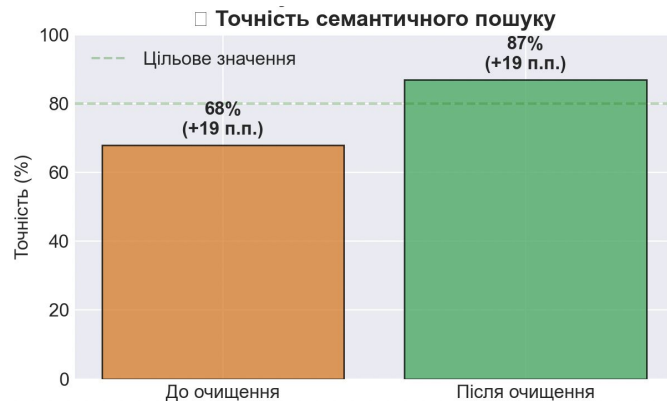
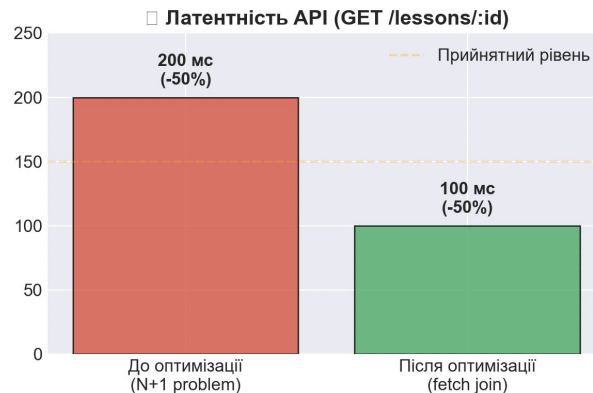
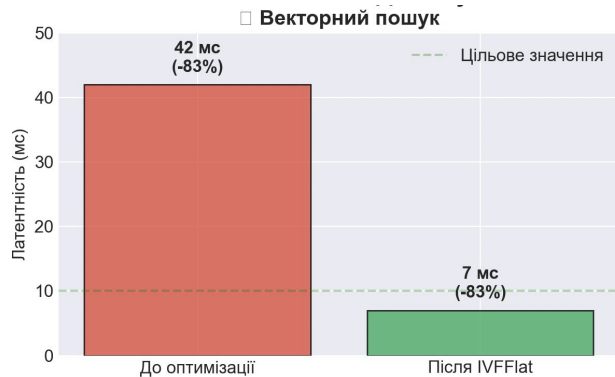
Алгоритм роботи векторного пошуку



Entity–Relationship діаграма моделі бази даних



Покращення продуктивності системи



Висновки

- Створено інтерактивну навчальну платформу з інтегрованим AI-асистентом (Qwen 2.5) та автоматичною перевіркою коду
- Впроваджено семантичний пошук на базі pgvector з відгуком до 10 мс та точністю 87%
- Досягнуто продуктивності 90-110 мс для API запитів через оптимізацію JOIN запитів та використання MapStruct
- Реалізовано надійний захист від DDoS (rate limiting з ефективністю 99.2%)
- Розроблено адміністративну панель для керування структурованим контентом (блоки, вкладеність, версіонування)
- Архітектура системи забезпечує горизонтальне масштабування та готовність до промислової експлуатації

Дякую за увагу